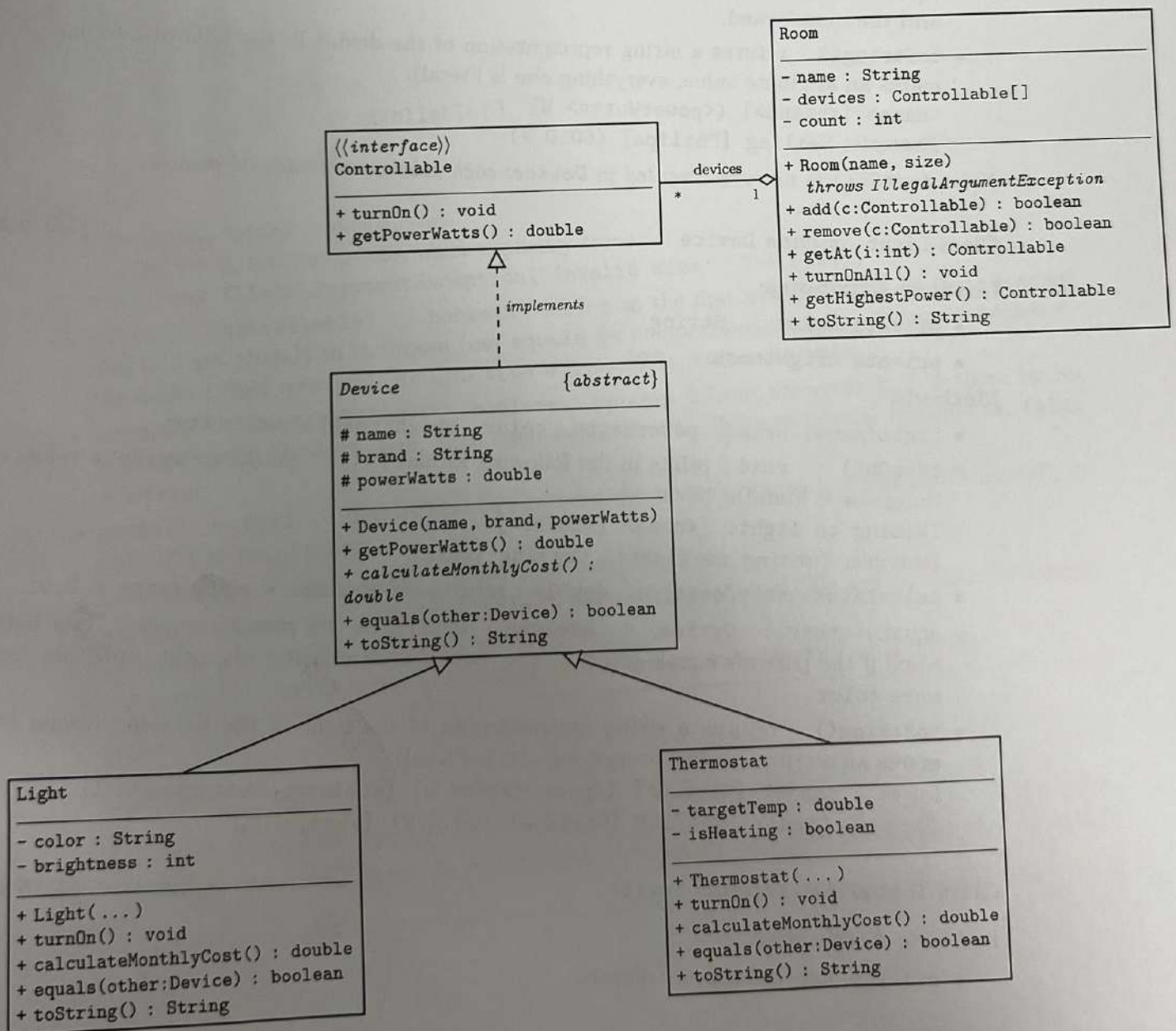
Instructions

- The use of any Java Collection or external library is **not allowed**.
- Open a new workspace before starting in your IDE.
- Write your name and ID at the top of every `.java` file as a comment.
- Submit all `.java` files to LMS. Any file not submitted receives **0**.
- Duration: **1 hour 50 minutes.** Total marks: **10.**

Scenario

We want to write a program that manages controllable smart-home devices (lights and thermostats) inside a room, using the following UML diagram:



[≈ 8.0 marks total]

Q1 : Implementation

Write the interface `Controllable` and the classes `Device`, `Light`, `Thermostat`, and `Room` as specified above. [2.0 marks]

Q2 : `RoomTest` (main method)

Write a class `RoomTest` containing a `main` method that demonstrates the functionality of your work. The `main` method should:

1. First, attempt to create a `Room` with an **invalid size** (e.g. `0`) inside a `try / catch` block, catching `IllegalArgumentException` and printing its message.
2. Then create a valid `Room` named `"Living Room"` with size 4.
3. Create the following devices and add them to the room:
   - `Light("Ceiling", "Philips", 60.0, "Warm", 80)`
   - `Thermostat("Main", "Nest", 5.0, 22.5, true)`
   - `Light("Desk", "Ikea", 15.0, "White", 100)`
   - `Thermostat("Bed", "Hwell", 5.0, 19.0, false)`
4. Print the room.
5. Attempt to `remove` a `Light` with the **same name and brand** as the Ceiling light but a **different color**, e.g. `new Light("Ceiling", "Philips", 60.0, "Cool", 80)`. Print the boolean result.
   Because `Light`'s `equals` override also checks `color`, the correct output is `false`. *If you forget to override `equals` and inherit only the parent's version (which checks just `name` and `brand`), this will incorrectly print `true` – which means your equals is wrong.*
6. Call `turnOnAll()` on the room.
7. Print the device with the **highest power** (using `getHighestPower()`).

Sample run

```
Invalid size: 0
----------------------------------------
Living Room (4):
Light: Ceiling [Philips] (60.0 W) [Warm, 80%]
Thermostat: Main [Nest] (5.0 W) [target: 22.5]
Light: Desk [Ikea] (15.0 W) [White, 100%]
Thermostat: Bed [Hwell] (5.0 W) [target: 19.0]
----------------------------------------
remove Light Ceiling/Philips (different color) -> false
----------------------------------------
Turning on light: Ceiling [Warm]
Activating thermostat: Main at 22.5
Turning on light: Desk [White]
Activating thermostat: Bed at 19.0
----------------------------------------
Highest power: Light: Ceiling [Philips] (60.0 W) [Warm, 80%]
```

- `private isHeating : boolean`

Methods:

- `Thermostat(name, brand, powerWatts, targetTemp, isHeating)` : constructor.
- `turnOn() : void` : prints in the following format (< . . . > marks an attribute value, everything else is literal):
  `Activating thermostat: <name> at <targetTemp>`
  Example: `Activating thermostat: Main at 22.5`
- `calculateMonthlyCost() : double` : returns `powerWatts * 0.50`.
- `equals(other : Device) : boolean` : **overrides** the parent's equals. Two thermostats are equal if the parent's equals returns `true`, `other` is an instance of `Thermostat`, **and** they have the same `targetTemp`.
- `toString()` : returns a string representation of the thermostat in the following format (< . . . > marks an attribute value, everything else is literal):
  `Thermostat: <name> [<brand>] (<powerWatts> W) [target: <targetTemp>]`
  Example: `Thermostat: Main [Nest] (5.0 W) [target: 22.5]`

Note: each child class ***must*** *override* `equals(Device)` *to add its own additional equality criteria. Children that do not override receive* ***0*** *marks for the equals criterion.*

[3.25 marks]

Class Room

Instance Attributes:

- `private name : String`
- `private devices : Controllable[]`
- `private count : int`

Methods:

- `Room(name, size)` : constructor.
  If `size <= 0`, the constructor must explicitly throw:
  `throw new IllegalArgumentException("Invalid size: " + size);`
- `add(c : Controllable) : boolean` : adds `c` to the first available space if there is space and `c` is not already in the room (use `equals` for comparison – polymorphism dispatches to the correct child override). Returns `true` if it succeeds, `false` otherwise.
- `remove(c : Controllable) : boolean` : removes `c` from the room if it is there by replacing it with the **last** item, then decrements `count`. Returns `true` if it succeeds, `false` otherwise.
- `getAt(i : int) : Controllable` : returns the device at index `i` (index starts from 0), or `null` if `i` is out of range.
- `turnOnAll() : void` : calls `turnOn()` on every device.
- `getHighestPower() : Controllable` : **recursively** returns the device with the largest `powerWatts`, or `null` if the room is empty.
- `toString()` : returns a string representation of the room in the following format:

```
name (count):
device1
device2
...
```

Note: `getHighestPower()` ***must*** *be implemented recursively. An iterative solution will receive* ***0*** *marks.*

Specification

[0.5 marks]

Interface `Controllable`

Methods (abstract):

- `turnOn()` : `void` : turns the device on.
- `getPowerWatts()` : `double` : returns the device's power consumption in watts.

[1.5 marks]

Abstract Class `Device` (*implements* `Controllable`)

Instance Attributes:

- `protected name` : `String`
- `protected brand` : `String`
- `protected powerWatts` : `double`

Methods:

- `Device(name, brand, powerWatts)` : constructor.
- `getPowerWatts()` : `double` : returns `powerWatts`.
- `calculateMonthlyCost()` : `double` : **abstract**.
- `equals(other : Device)` : `boolean` : two devices are equal if they have the same `name` **and** the same `brand`.
- `toString()` : returns a string representation of the device in the following format (`< . . . >` marks an attribute value, everything else is literal):
  `<name> [<brand>] (<powerWatts> W)`
  Example: `Ceiling [Philips] (60.0 W)`

Note: `turnOn()` is not implemented in `Device`; each subclass provides its own.

Class `Light` ***extends*** `Device` [1.5 marks]

Instance Attributes:

- `private color` : `String`
- `private brightness` : `int`

Methods:

- `Light(name, brand, powerWatts, color, brightness)` : constructor.
- `turnOn()` : `void` : prints in the following format (`< . . . >` marks an attribute value, everything else is literal):
  `Turning on light: <name> [<color>]`
  Example: `Turning on light: Ceiling [Warm]`
- `calculateMonthlyCost()` : `double` : returns `brightness * powerWatts * 0.01`.
- `equals(other : Device)` : `boolean` : **overrides** the parent's equals. Two lights are equal if the parent's equals returns `true`, `other` is an instance of `Light`, **and** they have the same `color`.
- `toString()` : returns a string representation of the light in the following format (`< . . . >` marks an attribute value, everything else is literal):
  `Light: <name> [<brand>] (<powerWatts> W) [<color>, <brightness>%]`
  Example: `Light: Ceiling [Philips] (60.0 W) [Warm, 80%]`

[1.25 marks]

Class `Thermostat` ***extends*** `Device`

Instance Attributes:

- `private targetTemp` : `double`